

```

1 !pip install -U tensorflow keras
2
3
4 ## Setup
5 import numpy as np
6 import matplotlib.pyplot as plt
7 import pandas as pd
8 import tensorflow as tf
9 import keras
10 import os
11
12 from sklearn.model_selection import train_test_split
13
14 os.environ["KERAS_BACKEND"] = "tensorflow"
15 keras.utils.set_random_seed(42)

```

```

1 # Not needed because we are using a public RoBERTa model
2 # from huggingface_hub import login
3 # from google.colab import userdata
4
5 # HF_TOKEN = userdata.get('HF_TOKEN') # Access the saved secret
6 # if HF_TOKEN:
7 #     login(token=HF_TOKEN)
8 #     print("Successfully logged in to Hugging Face!")
9 # else:
10 #     print("HF_TOKEN not set in Colab secrets.")

```

✓ Load Dataset

```
1 !pip install --upgrade kagglehub
```

```

Requirement already satisfied: kagglehub in /usr/local/lib/python3.12/dist-packages (0.3.13)
Collecting kagglehub
  Downloading kagglehub-1.0.0-py3-none-any.whl.metadata (40 kB)
    40.1/40.1 kB 3.5 MB/s eta 0:00:00
Collecting kagglesdk<1.0,>=0.1.14 (from kagglehub)
  Downloading kagglesdk-0.1.16-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kagglehub) (26.0)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from kagglehub) (6.0.3)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kagglehub) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kagglehub) (4.67.3)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kagglesdk<1.0,>=0.1.14->kagglehub) (4.25.3)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2.2.3)
Requirement already satisfied: certifi<=2024.7.4 in /usr/local/lib/python3.12/dist-packages (from requests->kagglehub) (2024.7.4)
  Downloading kagglehub-1.0.0-py3-none-any.whl (70 kB)
    70.6/70.6 kB 7.8 MB/s eta 0:00:00
  Downloading kagglesdk-0.1.16-py3-none-any.whl (160 kB)
    160.5/160.5 kB 10.1 MB/s eta 0:00:00
Installing collected packages: kagglesdk, kagglehub
Attempting uninstall: kagglehub
  Found existing installation: kagglehub 0.3.13
  Uninstalling kagglehub-0.3.13:
    Successfully uninstalled kagglehub-0.3.13
Successfully installed kagglehub-1.0.0 kagglesdk-0.1.16

```

```

1 import kagglehub
2
3 # Download latest version
4 path = kagglehub.dataset_download("eshummalik/socialbuzz-sentiment-analytics")
5
6 print("Path to dataset files:", path)

```

```

Using Colab cache for faster access to the 'socialbuzz-sentiment-analytics' dataset.
Path to dataset files: /kaggle/input/socialbuzz-sentiment-analytics

```

```

1 files = os.listdir(path)
2 print("Files found:", files)

```

```
Files found: ['sentimentdataset.csv']
```

```
1 df = pd.read_csv(os.path.join(path, "sentimentdataset.csv"))
2
3 print(df.head())
```

	Unnamed: 0.1	Unnamed: 0	\
0	0	0	
1	1	1	
2	2	2	
3	3	3	
4	4	4	

		Text	Sentiment	\
0		Enjoying a beautiful day at the park!	...	Positive
1		Traffic was terrible this morning.	...	Negative
2		Just finished an amazing workout! 🏋️	...	Positive
3		Excited about the upcoming weekend getaway!	...	Positive
4		Trying out a new recipe for dinner tonight.	...	Neutral

	Timestamp	User	Platform	\
0	2023-01-15 12:30:00	User123	Twitter	
1	2023-01-15 08:45:00	CommuterX	Twitter	
2	2023-01-15 15:45:00	FitnessFan	Instagram	
3	2023-01-15 18:20:00	AdventureX	Facebook	
4	2023-01-15 19:55:00	ChefCook	Instagram	

		Hashtags	Retweets	Likes	Country	\
0	#Nature #Park		15.0	30.0	USA	
1	#Traffic #Morning		5.0	10.0	Canada	
2	#Fitness #Workout		20.0	40.0	USA	
3	#Travel #Adventure		8.0	15.0	UK	
4	#Cooking #Food		12.0	25.0	Australia	

	Year	Month	Day	Hour
0	2023	1	15	12
1	2023	1	15	8
2	2023	1	15	15
3	2023	1	15	18
4	2023	1	15	19

▼ Demoji and Tokenize

```
1 !pip install demoji
2
3 import demoji
```

Collecting demoji
Downloading demoji-1.1.0-py3-none-any.whl.metadata (9.2 kB)
Downloading demoji-1.1.0-py3-none-any.whl (42 kB)
----- 42.9/42.9 kB 2.4 MB/s eta 0:00:00
Installing collected packages: demoji
Successfully installed demoji-1.1.0

```
1 df['cleaned_text'] = df['Text'].apply(lambda x: demoji.replace(x, "")) # without emoji
2 df['replaced_text'] = df['Text'].apply(lambda x: demoji.replace_with_desc(x)) # textualised emoji
3 df['Emoji'] = df['Text'].apply(lambda x: list(demoji.findall(x).values()))
4
5 ## df['Likes'].min() -> 10.0, df['Likes'].max() -> 80.0
6 df['Likes'] = df['Likes'].apply(lambda x: 'High' if x > 59 else 'Low')
7
8 df = df[['Likes', 'cleaned_text', 'replaced_text', 'Emoji']]
9
10 df.head()
```

	Likes	cleaned_text	replaced_text	Emoji
0	Low	Enjoying a beautiful day at the park! ...	Enjoying a beautiful day at the park! ...	😊
1	Low	Traffic was terrible this morning. ...	Traffic was terrible this morning. ...	😡
2	Low	Just finished an amazing workout! ...	Just finished an amazing workout! :flexed bic...	💪 [flexed biceps]
3	Low	Excited about the upcoming weekend getaway! ...	Excited about the upcoming weekend getaway! ...	😊
4	Low	Trying out a new recipe for dinner tonight. ...	Trying out a new recipe for dinner tonight. ...	😊

```
1 df.shape
```

(732, 4)

✓ Load the model and tokenizer

```
1 !pip install transformers==4.40.0
```

```
Collecting transformers==4.40.0
  Downloading transformers-4.40.0-py3-none-any.whl.metadata (137 kB)
    137.6/137.6 kB 6.1 MB/s eta 0:00:00
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (3.25.0)
Collecting huggingface-hub<1.0, >=0.19.3 (from transformers==4.40.0)
  Downloading huggingface_hub-0.36.2-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (2.0.2)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (24.1)
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (6.0.2)
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (2024.9.11)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (2.32.0)
Collecting tokenizers<0.20, >=0.19 (from transformers==4.40.0)
  Downloading tokenizers-0.19.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (6.7 kB)
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (0.4.5)
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers==4.40.0) (4.67.1)
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0, >=0.19.3) (2024.10.1)
Requirement already satisfied: hf-xet<2.0.0, >=1.1.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0, >=0.19.3) (1.1.5)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<1.0, >=0.19.3) (4.12.2)
Requirement already satisfied: charset-normalizer<4, >=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.0)
Requirement already satisfied: idna<4, >=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.10.1)
Requirement already satisfied: urllib3<3, >=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2025.1.1)
Downloading transformers-4.40.0-py3-none-any.whl (9.0 MB)
    9.0/9.0 MB 114.4 MB/s eta 0:00:00
Downloading huggingface_hub-0.36.2-py3-none-any.whl (566 kB)
    566.4/566.4 kB 53.8 MB/s eta 0:00:00
Downloading tokenizers-0.19.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (3.6 MB)
    3.6/3.6 MB 136.1 MB/s eta 0:00:00
Installing collected packages: huggingface-hub, tokenizers, transformers
  Attempting uninstall: huggingface-hub
    Found existing installation: huggingface_hub 1.6.0
    Uninstalling huggingface_hub-1.6.0:
      Successfully uninstalled huggingface_hub-1.6.0
  Attempting uninstall: tokenizers
    Found existing installation: tokenizers 0.22.2
    Uninstalling tokenizers-0.22.2:
      Successfully uninstalled tokenizers-0.22.2
  Attempting uninstall: transformers
    Found existing installation: transformers 5.0.0
    Uninstalling transformers-5.0.0:
      Successfully uninstalled transformers-5.0.0
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behavior is deprecated. Use the --upgrade flag to update the resolved set.
sentence-transformers 5.2.3 requires transformers<6.0.0, >=4.41.0, but you have transformers 4.40.0 which is incompatible.
Successfully installed huggingface-hub-0.36.2 tokenizers-0.19.1 transformers-4.40.0
```

```
1 from transformers import AutoTokenizer, AutoConfig
2 from transformers import TFRobertaModel
3
4 MODEL = f"cardiffnlp/twitter-roberta-base-sentiment-latest"
5 tokenizer = AutoTokenizer.from_pretrained(MODEL)
6 config = AutoConfig.from_pretrained(MODEL)
7
8 backbone = TFRobertaModel.from_pretrained(MODEL)
9
```

```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:949: FutureWarning: `resume_download` is deprecated and will be removed in version 0.11. The automatic resumption of downloads will be removed. Please call `resume_download` explicitly.
warnings.warn(
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret 'HF_TOKEN' does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/token)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
config.json: 100% 929/929 [00:00<00:00, 101kB/s]

vocab.json: 899k/? [00:00<00:00, 21.8MB/s]

merges.txt: 456k/? [00:00<00:00, 35.9MB/s]

special_tokens_map.json: 100% 239/239 [00:00<00:00, 32.6kB/s]

tf_model.h5: 100% 499M/499M [00:05<00:00, 235MB/s]
All model checkpoint layers were used when initializing TFRobertaModel.

All the layers of TFRobertaModel were initialized from the model checkpoint at cardiffnlp/twitter-roberta-base-sentiment
If your task is similar to the task the model of the checkpoint was trained on, you can already use TFRobertaModel for

```

```

1 def preprocess(text):
2     new_text = []
3     for t in str(text).split(" "): # Added str() just in case of NaNs
4         t = '@user' if t.startswith('@') and len(t) > 1 else t
5         t = 'http' if t.startswith('http') else t
6         new_text.append(t)
7     return " ".join(new_text)
8
9 df['cleaned_text'] = df['cleaned_text'].apply(preprocess)
10 df['replaced_text'] = df['replaced_text'].apply(preprocess)
11
12 # Tokenize 'cleaned_text' and store in DF
13
14 clean_encodings = tokenizer(df['cleaned_text'].tolist(), padding=True, truncation=True, return_tensors='pt')
15 df['cleaned_input_ids'] = list(clean_encodings['input_ids'])
16 df['cleaned_attention_mask'] = list(clean_encodings['attention_mask'])
17
18 # Tokenize 'replaced_text' and store in DF
19 replaced_encodings = tokenizer(df['replaced_text'].tolist(), padding=True, truncation=True, return_tensors='pt')
20 df['replaced_input_ids'] = list(replaced_encodings['input_ids'])
21 df['replaced_attention_mask'] = list(replaced_encodings['attention_mask'])
22
23 labels = (df["Likes"] == "High").astype(int).values

```

Asking to truncate to max_length but no maximum length is provided and the model has no predefined maximum length. De

```

1 cleaned_df = df[['cleaned_text', 'cleaned_input_ids', 'cleaned_attention_mask', 'Likes']]
2 replaced_df = df[['replaced_text', 'replaced_input_ids', 'replaced_attention_mask', 'Likes']]
3
4 cleaned_df.head()

```

	cleaned_text	cleaned_input_ids	cleaned_attention_mask	Likes
0	Enjoying a beautiful day at the park! ...	[tensor(0), tensor(16013), tensor(154), tensor(...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
1	Traffic was terrible this morning. ...	[tensor(0), tensor(11830), tensor(21), tensor(...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
2	Just finished an amazing workout! ...	[tensor(0), tensor(1801), tensor(1550), tensor(...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
3	Excited about the upcoming weekend getaway!	[tensor(0), tensor(22847), tensor(4560), tensor(...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low

Split Dataset

```

1 # Stratified by "Likes" (binary: 1,0)
2 labels = (df["Likes"] == "High").astype(int)
3
4 # Train set index
5 train_idx, temp_idx = train_test_split(
6     df.index,
7     test_size=0.2,

```

```
8 stratify=labels,
9 random_state=42
10 )
11
12 # Validation set index + Test set index
13 val_idx, test_idx = train_test_split(
14     temp_idx,
15     test_size=0.5,
16     stratify=labels[temp_idx],
17     random_state=42
18 )
19
20
21 # cleaned_df dataset
22 c_train_df = cleaned_df.loc[train_idx]
23 c_val_df = cleaned_df.loc[val_idx]
24 c_test_df = cleaned_df.loc[test_idx]
25
26 # replaced_df dataset
27 r_train_df = replaced_df.loc[train_idx]
28 r_val_df = replaced_df.loc[val_idx]
29 r_test_df = replaced_df.loc[test_idx]
30
31 # labels
32 y_train = labels.loc[train_idx].values
33 y_val = labels.loc[val_idx].values
34 y_test = labels.loc[test_idx].values
```

```
1 c_train_df.head()
```

	cleaned_text	cleaned_input_ids	cleaned_attention_mask	Likes
490	Standing before the grandeur of the Eiffel Tow...	[tensor(0), tensor(45150), tensor(137), tensor...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
155	Empowerment through learning a new skill. ...	[tensor(0), tensor(3676), tensor(11017), tenso...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
341	Curiosity sparked by exploring a mysterious an...	[tensor(0), tensor(42090), tensor(45226), tens...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
...	Rocking out at a Guns N' Roses show, the	[tensor(0), tensor(500), tensor(12088),	[tensor(1), tensor(1), tensor(1), tensor(1),	.

```
1 r_train_df.head()
```

	replaced_text	replaced_input_ids	replaced_attention_mask	Likes
490	Standing before the grandeur of the Eiffel Tow...	[tensor(0), tensor(45150), tensor(137), tensor...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
155	Empowerment through learning a new skill. ...	[tensor(0), tensor(3676), tensor(11017), tenso...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
341	Curiosity sparked by exploring a mysterious an...	[tensor(0), tensor(42090), tensor(45226), tens...	[tensor(1), tensor(1), tensor(1), tensor(1), t...	Low
...	Rocking out at a Guns N' Roses show, the	[tensor(0), tensor(500), tensor(12088),	[tensor(1), tensor(1), tensor(1), tensor(1),	.

Load a Pre-Trained Model

Finetuning

```

1 # Compute weights to correct imbalanced dataset
2 # from sklearn.utils.class_weight import compute_class_weight
3 # import numpy as np
4
5 # classes = np.unique(y_train)
6
7 # weights = compute_class_weight(
8 #     class_weight="balanced",
9 #     classes=classes,
10 #     y=y_train
11 # )
12
13 # class_weights = dict(zip(classes, weights))
14 # print(class_weights)

```

```
1 import keras_hub
```

```

1 # Check the actual lengths of tokenized rows
2
3 # cleaned
4 lengths = [len(x) for x in df['cleaned_input_ids']]
5 print(f"Max length in your data: {max(lengths)}")
6 print(f"Average length: {sum(lengths)/len(lengths)}")
7
8 # replaced
9 lengths = [len(x) for x in df['replaced_input_ids']]
10 print(f"Max length in your data: {max(lengths)}")
11 print(f"Average length: {sum(lengths)/len(lengths)}")

```

```

Max length in your data: 41
Average length: 41.0
Max length in your data: 41
Average length: 41.0

```

```
1 !pip install keras_tuner
```

```

Collecting keras_tuner
  Downloading keras_tuner-1.4.8-py3-none-any.whl.metadata (5.6 kB)
Requirement already satisfied: keras in /usr/local/lib/python3.12/dist-packages (from keras_tuner) (3.13.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from keras_tuner) (26.0)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from keras_tuner) (2.32.4)
Collecting kt_legacy
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: grpcio in /usr/local/lib/python3.12/dist-packages (from keras_tuner) (1.78.0)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from keras_tuner) (5.29.6)
Requirement already satisfied: typing-extensions~=4.12 in /usr/local/lib/python3.12/dist-packages (from grpcio->keras_tuner) (4.12.0)
Requirement already satisfied: absl-py in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (0.1.0)
Requirement already satisfied: h5py in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (3.16.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (0.19.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.12/dist-packages (from keras->keras_tuner) (0.5.4)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->keras_tuner) (3.4.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->keras_tuner) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests->keras_tuner) (2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->keras_tuner) (2025.1.1)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras_tuner) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras_tuner) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras_tuner) (0.1.2)
Downloading keras_tuner-1.4.8-py3-none-any.whl (129 kB)
129.4/129.4 KB 6.9 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt_legacy, keras_tuner
Successfully installed keras_tuner-1.4.8 kt_legacy-1.0.5

```

```

1 class RobertaLayer(keras.layers.Layer):
2     def __init__(self, backbone, **kwargs):
3         super().__init__(**kwargs)
4         self.backbone = backbone
5
6     def call(self, inputs):
7         input_ids, attention_mask = inputs
8         outputs = self.backbone(
9             input_ids=input_ids,
10            attention_mask=attention_mask,
11            training=False

```

```

12     )
13     return outputs.last_hidden_state[:, 0, :]

```

```

1 import keras_tuner as kt
2
3 def build_model(hp):
4     # Define hyper-parameters
5     units = hp.Choice('units', values=[32, 64, 128])
6     dropout = hp.Choice('dropout', values=[0.1, 0.2, 0.3])
7     lr = hp.Choice('learning_rate', values=[1e-3, 1e-4, 1e-5])
8
9     backbone.trainable = True
10
11     input_ids = keras.layers.Input(shape=(41,), dtype=tf.int32, name="input_ids")
12     attention_mask = keras.layers.Input(shape=(41,), dtype=tf.int32, name="attention_mask")
13
14     cls_output = RobertaLayer(backbone)([input_ids, attention_mask])
15
16     x = keras.layers.Dense(units, activation='relu')(cls_output)
17     x = keras.layers.Dropout(dropout)(x)
18     out = keras.layers.Dense(1, activation='sigmoid')(x)
19
20     model = keras.Model(inputs=[input_ids, attention_mask], outputs=out)
21     model.compile(
22         optimizer=keras.optimizers.Adam(learning_rate=lr),
23         loss='binary_crossentropy',
24         metrics=['accuracy']
25     )
26     return model
27
28 # GridSearch
29 tuner_C = kt.GridSearch(
30     hypermodel=build_model,
31     objective='val_accuracy',
32     max_trials=None,
33     directory='my_dir_C',
34     project_name='keras_grid_search_C'
35 )

```

```

1 ### MODEL 1: Without Emoji
2 # Prepare data for cleaned
3 x_train_C = {
4     "input_ids": tf.constant([t.numpy() for t in c_train_df["cleaned_input_ids"]], dtype=tf.int32),
5     "attention_mask": tf.constant([t.numpy() for t in c_train_df["cleaned_attention_mask"]], dtype=tf.int32)
6 }
7 x_val_C = {
8     "input_ids": tf.constant([t.numpy() for t in c_val_df["cleaned_input_ids"]], dtype=tf.int32),
9     "attention_mask": tf.constant([t.numpy() for t in c_val_df["cleaned_attention_mask"]], dtype=tf.int32)
10 }
11
12 tuner_C.search(
13     x=x_train_C,
14     y=y_train,
15     epochs=5,
16     batch_size=32,
17     validation_data=(x_val_C, y_val)
18 )
19
20 # Check best parameters
21 best_hp = tuner_C.get_best_hyperparameters(1)[0]
22 print("Best units:", best_hp.get('units'))
23 print("Best dropout:", best_hp.get('dropout'))
24 print("Best lr:", best_hp.get('learning_rate'))

```

Trial 27 Complete [00h 00m 42s]
val_accuracy: 0.8493150472640991

Best val_accuracy So Far: 0.8493150472640991
Total elapsed time: 00h 17m 49s
Best units: 32
Best dropout: 0.1
Best lr: 0.001

```

1 # Make the best model
2 cleaned_best_model = tuner_C.get_best_models(num_models=1)[0]
3

```

```

4 #fit
5 historyC = cleaned_best_model.fit(
6     x_train_C,
7     y_train,
8     validation_data=(x_val_C, y_val),
9     epochs=5,
10    batch_size=32
11 )
12
13 # Prepare test data
14 x_test_C = {
15     "input_ids":      tf.constant([t.numpy() for t in c_test_df["cleaned_input_ids"]], dtype=tf.int32),
16     "attention_mask": tf.constant([t.numpy() for t in c_test_df["cleaned_attention_mask"]], dtype=tf.int32)
17 }
18
19 # Evaluate
20 loss, accuracy = cleaned_best_model.evaluate(x_test_C, y_test)
21 print(f"Test Loss: {loss:.4f}")
22 print(f"Test Accuracy: {accuracy:.4f}")
23
24 # Estimate
25 predictions = cleaned_best_model.predict(x_test_C)
26 pred_labels = (predictions > 0.5).astype(int)

```

```

Epoch 1/5
/usr/local/lib/python3.12/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping variable loading for saveable.load_own_variables(weights_store.get(inner_path))
19/19 ━━━━━━━━━━━ 29s 774ms/step - accuracy: 0.8462 - loss: 0.4309 - val_accuracy: 0.8493 - val_loss: 0.4297
Epoch 2/5
19/19 ━━━━━━━━━━━ 2s 85ms/step - accuracy: 0.8410 - loss: 0.4124 - val_accuracy: 0.8493 - val_loss: 0.4201
Epoch 3/5
19/19 ━━━━━━━━━━━ 2s 85ms/step - accuracy: 0.8479 - loss: 0.4107 - val_accuracy: 0.8493 - val_loss: 0.3875
Epoch 4/5
19/19 ━━━━━━━━━━━ 2s 86ms/step - accuracy: 0.8462 - loss: 0.3784 - val_accuracy: 0.8493 - val_loss: 0.3882
Epoch 5/5
19/19 ━━━━━━━━━━━ 2s 86ms/step - accuracy: 0.8479 - loss: 0.3695 - val_accuracy: 0.8493 - val_loss: 0.3890
3/3 ━━━━━━━━━━━ 3s 1s/step - accuracy: 0.8514 - loss: 0.4444
Test Loss: 0.4444
Test Accuracy: 0.8514
3/3 ━━━━━━━━━━━ 9s 2s/step

```

```
1 cleaned_best_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	(None, 41)	0	–
attention_mask (InputLayer)	(None, 41)	0	–
roberta_layer (RobertaLayer)	(None, 768)	0	input_ids[0][0], attention_mask[0]...
dense (Dense)	(None, 32)	24,608	roberta_layer[0]...
dropout (Dropout)	(None, 32)	0	dense[0][0]
dense_1 (Dense)	(None, 1)	33	dropout[0][0]

Total params: 73,925 (288.77 KB)
 Trainable params: 24,641 (96.25 KB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 49,284 (192.52 KB)

****Note:** The parameter count in model.summary() may show 0 for the backbone layer. This is a known limitation of Keras 3.x, which cannot track weights of external TF models. However, the backbone weights are still included in training and will be updated normally.

```

1 ### MODEL 2: Without Emoji
2 import keras_tuner as kt
3 # GridSearch
4 tuner_R = kt.GridSearch(
5     hypermodel=build_model,
6     objective='val_accuracy',
7     max_trials=None,
8     directory='my_dir_R',

```



```

9     project_name='keras_grid_search_R'
10 )
11
12 # Prepare data for cleaned
13 x_train_R = {
14     "input_ids":      tf.constant([t.numpy() for t in r_train_df["replaced_input_ids"]], dtype=tf.int32),
15     "attention_mask": tf.constant([t.numpy() for t in r_train_df["replaced_attention_mask"]], dtype=tf.int32)
16 }
17 x_val_R = {
18     "input_ids":      tf.constant([t.numpy() for t in r_val_df["replaced_input_ids"]], dtype=tf.int32),
19     "attention_mask": tf.constant([t.numpy() for t in r_val_df["replaced_attention_mask"]], dtype=tf.int32)
20 }
21
22 tuner_R.search(
23     x=x_train_R,
24     y=y_train,
25     epochs=5,
26     batch_size=32,
27     validation_data=(x_val_R, y_val)
28 )
29
30 # Check best parameters
31 best_hp = tuner_R.get_best_hyperparameters(1)[0]
32 print("Best units:", best_hp.get('units'))
33 print("Best dropout:", best_hp.get('dropout'))
34 print("Best lr:", best_hp.get('learning_rate'))

```

Trial 27 Complete [00h 00m 46s]
val_accuracy: 0.8493150472640991

Best val_accuracy So Far: 0.8630136847496033
Total elapsed time: 00h 19m 35s
Best units: 64
Best dropout: 0.1
Best lr: 1e-05

```

1 # Make the best model
2 replaced_best_model = tuner_R.get_best_models(num_models=1)[0]
3
4 # Prepare test data
5 x_test_R = {
6     "input_ids":      tf.constant([t.numpy() for t in r_test_df["replaced_input_ids"]], dtype=tf.int32),
7     "attention_mask": tf.constant([t.numpy() for t in r_test_df["replaced_attention_mask"]], dtype=tf.int32)
8 }
9
10 #fit
11 historyR = replaced_best_model.fit(
12     x_train_R,
13     y_train,
14     validation_data=(x_val_R, y_val),
15     epochs=5,
16     batch_size=32
17 )
18
19 # Evaluate
20 loss, accuracy = replaced_best_model.evaluate(x_test_R, y_test)
21 print(f"Test Loss: {loss:.4f}")
22 print(f"Test Accuracy: {accuracy:.4f}")
23
24 # Estimate
25 predictions = replaced_best_model.predict(x_test_R)
26 pred_labels = (predictions > 0.5).astype(int)

```

Epoch 1/5
19/19 ————— 29s 763ms/step - accuracy: 0.8256 - loss: 0.4704 - val_accuracy: 0.8630 - val_loss: 0.4331
Epoch 2/5
19/19 ————— 2s 87ms/step - accuracy: 0.8376 - loss: 0.4344 - val_accuracy: 0.8493 - val_loss: 0.4230
Epoch 3/5
19/19 ————— 2s 89ms/step - accuracy: 0.8393 - loss: 0.4343 - val_accuracy: 0.8493 - val_loss: 0.4169
Epoch 4/5
19/19 ————— 2s 89ms/step - accuracy: 0.8376 - loss: 0.4412 - val_accuracy: 0.8493 - val_loss: 0.4134
Epoch 5/5
19/19 ————— 2s 91ms/step - accuracy: 0.8410 - loss: 0.4333 - val_accuracy: 0.8493 - val_loss: 0.4108
3/3 ————— 3s 1s/step - accuracy: 0.8514 - loss: 0.4258
Test Loss: 0.4258
Test Accuracy: 0.8514
3/3 ————— 9s 3s/step

```
1 replaced_best_model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_ids (InputLayer)	(None, 41)	0	–
attention_mask (InputLayer)	(None, 41)	0	–
roberta_layer (RobertaLayer)	(None, 768)	0	input_ids[0][0], attention_mask[0]...
dense (Dense)	(None, 64)	49,216	roberta_layer[0]...
dropout (Dropout)	(None, 64)	0	dense[0][0]
dense_1 (Dense)	(None, 1)	65	dropout[0][0]

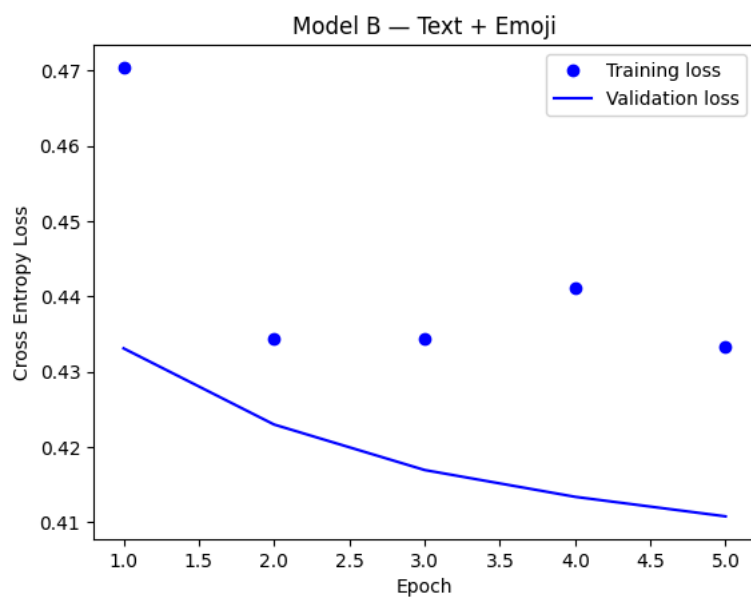
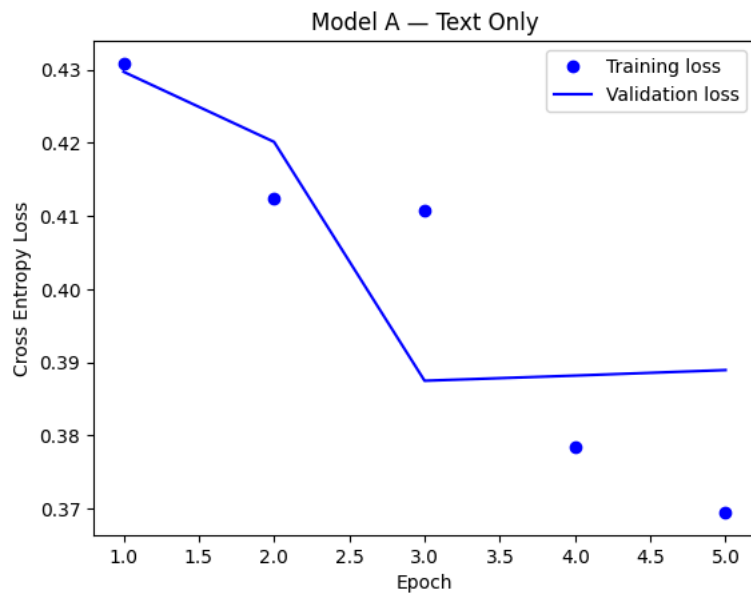
Total params: 147,845 (577.52 KB)
Trainable params: 49,281 (192.50 KB)
Non-trainable params: 0 (0.00 B)
Optimizer params: 98,564 (385.02 KB)

****Note:** The parameter count in model.summary() may show 0 for the backbone layer. This is a known limitation of Keras 3.x, which cannot track weights of external TF models. However, the backbone weights are still included in training and will be updated normally.

Loss Curve

```
1 # Function for plotting loss curve
2
3 def plot_loss(history, title):
4
5     train_loss = history.history["loss"]
6     val_loss = history.history["val_loss"]
7
8     epochs = range(1, len(train_loss)+1)
9
10    plt.figure()
11
12    plt.plot(epochs, train_loss, "bo", label="Training loss")
13    plt.plot(epochs, val_loss, "b", label="Validation loss")
14
15    plt.title(title)
16    plt.xlabel("Epoch")
17    plt.ylabel("Cross Entropy Loss")
18
19    plt.legend()
20    plt.show()
```

```
1 plot_loss(historyC, "Model A – Text Only")
2 plot_loss(historyR, "Model B – Text + Emoji")
```



✓ Predictions

```
1 # predict
2 y_prob_C = cleaned_best_model.predict(x_test_C)
3 y_pred_C = (y_prob_C > 0.5).astype(int)
4
5 # evaluate
6 cleaned_best_model.evaluate(x_train_C, y_train)
7 cleaned_best_model.evaluate(x_test_C, y_test)
```

3/3 ————— 1s 325ms/step
 19/19 ————— 2s 75ms/step - accuracy: 0.8462 - loss: 0.3520
 3/3 ————— 0s 66ms/step - accuracy: 0.8514 - loss: 0.4444
 [0.4443517029285431, 0.8513513803482056]

```
1 # predict
2 y_prob_R = replaced_best_model.predict(x_test_R)
3 y_pred_R = (y_prob_R > 0.5).astype(int)
4
5 # evaluate
6 replaced_best_model.evaluate(x_train_R, y_train)
7 replaced_best_model.evaluate(x_test_R, y_test)
```

3/3 ————— 0s 69ms/step
 19/19 ————— 1s 76ms/step - accuracy: 0.8462 - loss: 0.4203

3/3 0s 73ms/step - accuracy: 0.8514 - loss: 0.4258
[0.42580386996269226, 0.8513513803482056]

Evaluation metrics

```
1 from sklearn.metrics import f1_score, roc_auc_score, classification_report
2
3 f1_C = f1_score(y_test, y_pred_C)
4 roc_C = roc_auc_score(y_test, y_prob_C)
5
6 print("Cleaned Model")
7 print("F1-score:", f1_C)
8 print("ROC-AUC:", roc_C)
9
10 print("\nClassification Report")
11 print(classification_report(y_test, y_pred_C, target_names=["Low", "High"]))
```

Cleaned Model
F1-score: 0.0
ROC-AUC: 0.5613275613275615

Classification Report				
	precision	recall	f1-score	support
Low	0.85	1.00	0.92	63
High	0.00	0.00	0.00	11
accuracy			0.85	74
macro avg	0.43	0.50	0.46	74
weighted avg	0.72	0.85	0.78	74

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```
1 from sklearn.metrics import f1_score, roc_auc_score, classification_report
2
3 f1_R = f1_score(y_test, y_pred_R)
4 roc_R = roc_auc_score(y_test, y_prob_R)
5
6 print("Cleaned Model")
7 print("F1-score:", f1_R)
8 print("ROC-AUC:", roc_R)
9
10 print("\nClassification Report")
11 # correct: y_pred_R
12 print(classification_report(y_test, y_pred_R, target_names=["Low", "High"]))
```

Cleaned Model
F1-score: 0.0
ROC-AUC: 0.5468975468975468

Classification Report				
	precision	recall	f1-score	support
Low	0.85	1.00	0.92	63
High	0.00	0.00	0.00	11
accuracy			0.85	74
macro avg	0.43	0.50	0.46	74
weighted avg	0.72	0.85	0.78	74

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))

```
1 results = pd.DataFrame({
2
3     "Model": ["Text Only", "Text + Emoji"],
4
```

```

5     "F1 Score": [f1_C, f1_R],
6
7     "ROC-AUC": [roc_C, roc_R]
8
9 })
10
11 results

```

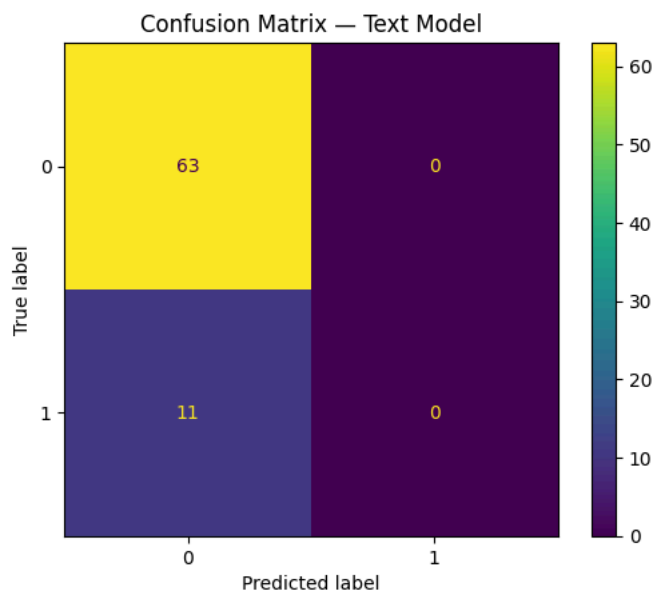
	Model	F1 Score	ROC-AUC
0	Text Only	0.0	0.561328
1	Text + Emoji	0.0	0.546898

Confusion Matrix

```

1 from sklearn.metrics import ConfusionMatrixDisplay
2 import matplotlib.pyplot as plt
3
4 ConfusionMatrixDisplay.from_predictions(
5     y_test,
6     y_pred_C
7 )
8
9 plt.title("Confusion Matrix - Text Model")
10 plt.show()

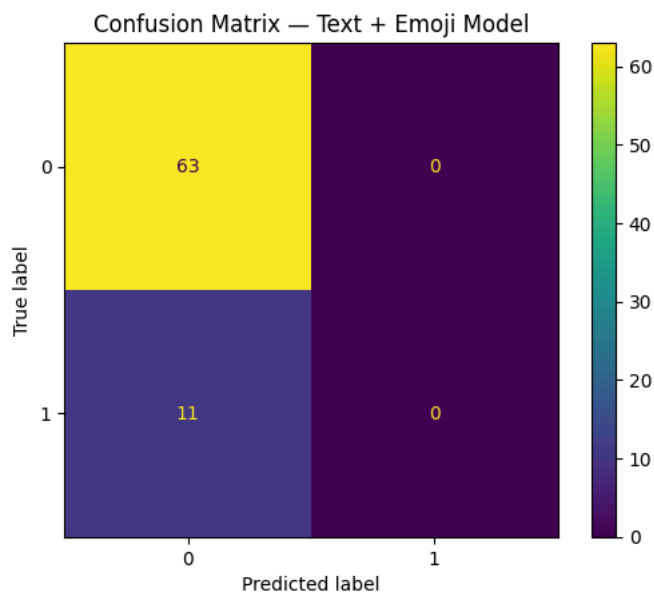
```



```

1 ConfusionMatrixDisplay.from_predictions(
2     y_test,
3     y_pred_R
4 )
5
6 plt.title("Confusion Matrix - Text + Emoji Model")
7 plt.show()

```



✓ SHAP explanation

```

1 import shap
2
3 def predict_fn(texts):
4     encoded = tokenizer(
5         list(texts),
6         max_length=41,
7         padding="max_length",
8         truncation=True,
9         return_tensors="tf"
10    )
11    probs = cleaned_best_model.predict({
12        "input_ids": encoded["input_ids"],
13        "attention_mask": encoded["attention_mask"]
14    })
15    return probs # shape (N, 1)
16
17 explainer_C = shap.Explainer(
18     predict_fn,
19     shap.maskers.Text(tokenizer)
20 )
21
22 # train 텍스트 원본으로 넘기기
23 shap_values_C = explainer_C(list(c_train_df["cleaned_text"][:20]))
24
25 shap.plots.text(shap_values_C[0])

```

```

1/1 ————— 4s 4s/step
1/1 ————— 0s 49ms/step
1/1 ————— 3s 3s/step
1/1 ————— 3s 3s/step

```

```

1/1 ————— 0s 47ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 47ms/step
1/1 ————— 0s 46ms/step
1/1 ————— 0s 58ms/step
1/1 ————— 0s 60ms/step
1/1 ————— 0s 58ms/step
1/1 ————— 0s 59ms/step
1/1 ————— 0s 60ms/step
1/1 ————— 0s 58ms/step
1/1 ————— 0s 60ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 60ms/step
1/1 ————— 0s 61ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 61ms/step
1/1 ————— 0s 64ms/step
1/1 ————— 0s 64ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 61ms/step
1/1 ————— 0s 63ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 62ms/step
1/1 ————— 0s 64ms/step

```

```

1 def predict_fn_replaced(texts):
2     encoded = tokenizer(
3         list(texts),
4         max_length=41,
5         padding="max_length",
6         truncation=True,
7         return_tensors="tf"
8     )
9     probs = replaced_best_model.predict({
10         "input_ids": encoded["input_ids"],
11         "attention_mask": encoded["attention_mask"]
12     })
13     return probs
14
15 explainer_R = shap.Explainer(
16     predict_fn_replaced,
17     shap.maskers.Text(tokenizer)
18 )
19
20 shap_values_R = explainer_R(list(r_train_df["replaced_text"][:20]))
21
22 shap.plots.text(shap_values_R[0])

```

```

1/1 ————— 0s 62ms/step

```